Predicting bina.az Sale Prices and Price Tiers: A From-Scratch Decision Tree and Pegasos SVM

Mammadli, Esmira Mammadova, Gunash Muradov, Ali Mursalov, Punhan

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We predict the sale price of bina.az real-estate listings (Task A, regression, in AZN) and a derived binary price tier, premium versus standard around the training median (Task B, classification). A decision tree (Gini, entropy and MSE criteria) and a Pegasos support-vector machine with random Fourier features (RFF) are implemented from scratch in NumPy and benchmarked against scikit-learn models with matched settings. Hyperparameters are chosen on a validation split; the test split is scored once. On 64,350 cleaned listings our tree reaches a test RMSE of 358,672 AZN on Task A and an F1 of 0.919 on Task B, within 0.1% RMSE and 0.002 F1 of the scikit-learn tree. Our SVM reaches F1 0.838 (ROC-AUC 0.924), equal to scikit-learn's linear SVC (0.839), and trains about 36 times faster per fold in our cross-validation runs. The tree is better on F1 even though the SVM has the higher ROC-AUC. Errors concentrate on the most expensive listings (the top 1% of prices are under-predicted by 1.5M AZN on average) and on sparsely represented districts. Optional bonus models (ridge/SVR, random forest, AdaBoost, clustering) are also reported. Code: `github.com/Gunas12/ML_Final_Project` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, regression, classification, real-estate price prediction.

I. Introduction

Sale prices of apartments, houses and land plots in Azerbaijan vary over several orders of magnitude, and a model that estimates a fair price from a listing's location, size and condition is useful to buyers and sellers. We use a public bina.az scrape and study two tasks: predicting the price (Task A) and predicting whether a listing is above the median training price (Task B). The aim of the project is not a new model but a careful, reproducible comparison of two classic learners that we implement ourselves.

Our contributions are: (i) a NumPy decision tree for classification and regression and a NumPy Pegasos SVM with random Fourier features, both tested against scikit-learn; (ii) a leakage-aware data pipeline with a fixed split and a single test evaluation; (iii) a benchmark with matched scikit-learn baselines, 5-fold cross-validation and timing; (iv) an error analysis that shows where the models fail; and (v) several bonus models built on the same code. One command, `python -m src.run_all`, regenerates every number, table and figure; results are deterministic for seed 42.

II. Related Work / Background

Classification and regression trees recursively partition the feature space using threshold rules. At each node, the split is selected to reduce task-specific impurity: Gini impurity or entropy for classification and mean-squared error for regression [1]. This produces interpretable piecewise-constant predictions, while tree depth and minimum leaf size control the bias–variance trade-off. Our NumPy implementation uses these criteria for the binary price-tier and continuous price tasks.

Support-vector machines separate classes with a maximum-margin boundary; the soft-margin formulation trades margin width against hinge-loss violations [2]. Pegasos optimizes the primal objective with stochastic subgradient updates and a learning rate inversely proportional to iteration and regularization strength [3]. For non-linear decision boundaries, random Fourier features approximate the RBF kernel with an explicit finite-dimensional mapping [4].

III. Data and Preprocessing

We use the Kaggle *binaaz-sale-project* dataset, whose supplied CSV contains 100,775 scraped rows and 51 columns; prices are in AZN. The rows describe only 64,454 distinct listings, because a listing is scraped several times. We keep the most recently scraped row of each listing. Left in, the same listing appeared in both train and test and inflated the test score. Plausibility filters then remove 104 rows with prices outside $[5{,}000; 20{,}000{,}000]$ AZN or areas outside $[8; 10^6]$ m^2 (typos such as an 11 AZN flat), leaving **64,350 listings**. Valid expensive listings are kept rather than capped. Areas given in *sot* are converted to m^2; floor and building floors are parsed from the floor field. Missing numeric values are imputed with training medians and missing categories become `Unknown` before one-hot encoding, giving 171 features.

We exclude identifiers, URLs, free text, scrape timestamps, view counts and currency labels. `unit_price` and `total_price` are removed as well: they are derived from the target and would leak it into both tasks. For Task B a listing is premium (1) only when its price is strictly above the *training* median (210,000 AZN); ties are standard (0), and the same threshold labels validation and test rows. The seed-42 split is stratified by tier: 70% train, 15% validation and 15% test (45,045, 9,652 and 9,653 listings). Standardisation is fitted on training rows only.

IV. Method

A. Decision tree (from scratch)

A node splits samples with $x_j \le \tau$ to the left. The impurity is Gini $1 - \sum_k p_k^2$, entropy $-\sum_k p_k \log p_k$, or, for regression, the variance of the targets (MSE). For each

Table 1. Dataset summary after cleaning.

Property	Value
Raw rows	100,775
Distinct listings	64,454
After plausibility filters	64,350
Features used	171
Train / Val / Test listings	45,045 / 9,652 / 9,653
Tier threshold (train median)	210,000 AZN

feature we sort the values once and evaluate every midpoint between consecutive distinct values using cumulative class counts (or cumulative sums and sums of squares for regression), so a split search costs $O(n \log n)$ per feature. The split with the largest weighted impurity decrease $\frac{n}{N}\big(I(\text{parent}) - \frac{n_L}{n} I(\text{left}) - \frac{n_R}{n} I(\text{right})\big)$ is taken. Growth is recursive and stops on `max_depth`, `min_samples_split`, `min_samples_leaf` or `min_impurity_decrease`. A leaf predicts the class counts (normalised for `predict_proba`) or the mean target. Feature importances are the summed impurity decreases. The same class also supports random feature subsets (`max_features`), which the forest bonus uses.

B. Support-vector machine (from scratch)

For the binary price-tier task, the two original class labels are mapped deterministically to $\{-1, +1\}$ during fitting. Predictions are mapped back to the original labels before evaluation. Our NumPy implementation minimises the soft-margin primal objective

$$J(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|_2^2 + \frac{1}{n}\sum_{i=1}^{n} \max\Big(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\Big). \quad (1)$$

Training uses a single-sample, Pegasos-style stochastic subgradient variant, without a projection step. At iteration t, one training example is sampled using a fixed random seed and the learning rate is $\eta_t = 1/(\lambda t)$. The weight vector is always shrunk by $1 - \eta_t \lambda$. If the sampled margin $y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ is below one, the update is

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\mathbf{w} + \eta_t y_i \mathbf{x}_i, \quad (2)$$

$$b \leftarrow b + \eta_t y_i. \quad (3)$$

Otherwise only the shrinkage is applied. The bias is not regularised, and inputs must be standardised. The decision function is $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$; samples with $y_i f(\mathbf{x}_i) \le 1$ are reported as margin-violating, and the geometric margin width is $2/\|\mathbf{w}\|_2$.

For a non-linear extension we approximate the RBF kernel with random Fourier features [4]:

$$\phi(\mathbf{x}) = \sqrt{\tfrac{2}{D}}\big[\cos(\omega_1^\top \mathbf{x} + b_1), \ldots, \cos(\omega_D^\top \mathbf{x} + b_D)\big], \quad (4)$$

with $\omega_j \sim \mathcal{N}(\mathbf{0}, 2\gamma \mathbf{I})$ and $b_j \sim \mathcal{U}[0, 2\pi]$. The map is drawn once, fitted on the training data, and reused unchanged for validation and test. Linear Pegasos is then trained in the transformed space. D trades approximation quality against cost; γ sets the locality of the approximated kernel.

V. Experimental Setup

Metrics. Task A: RMSE, MAE and R^2 in AZN, plus RMSE and R^2 on log price, because prices are heavy-tailed. Task B: accuracy, precision, recall, F1, ROC-AUC and PR-AUC. All metrics are NumPy implementations in `src/evaluate.py`.

Protocol. Every hyperparameter is chosen on the validation split. The test split is scored exactly once, after the final configurations are fixed. Cross-validation (5 folds; stratified by tier for Task B) uses only train+validation rows, with scaling fitted inside each fold. The seed is 42 throughout. The tree search covers depths $\{1, 2, 3, 4, 5, 6, 8, 10, 12, 15, \text{none}\}$ and minimum leaf sizes $\{1, 2, 5, 10, 20, 50, 100, 200\}$; for Task B Gini and entropy are compared, each at its own best setting.

SVM protocol. The linear search uses $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$. The RFF search is a *joint* grid over $\lambda \in \{10^{-5}, 3 \cdot 10^{-5}, 10^{-4}, 3 \cdot 10^{-4}\}$ and $\gamma \in \{0.003, 0.01, 0.03, 0.1\}$, because the best λ for the linear model need not suit the feature map. Validation F1 selects the model; ties prefer stronger regularisation, and linear wins a tie between families. The full run uses $D = 1024$ components and 400,000 single-example updates per candidate, about 8.9 passes over the 45,045 training rows. The `-fast` configuration is only for development.

Baselines. Each scikit-learn model minimises the same objective as ours (`src/baselines.py`). Trees use the same criterion, depth and leaf settings. For the linear SVC, $C = 1/(\lambda n)$ makes its objective a rescaling of ours. `SGDClassifier` (hinge) uses $\alpha = \lambda$, and `RBFSampler`+SGD is the matched pipeline for RFF. Because the exact kernel SVC is $O(n^2)$, we compare with linear SVC. Fit times are wall-clock means per CV fold on one laptop and should be read as ratios, not absolutes.

VI. Results

A. Results A: decision tree

Validation selects depth 8 and a minimum leaf size of 10 for Task A (125 leaves). Test RMSE is 358,672 AZN ($R^2 = 0.661$) against 358,999 for scikit-learn's tree. An unlimited-depth tree memorises the training data (train RMSE 3,951 vs. validation RMSE 535,870), so depth acts as the main regulariser. The 5-fold CV RMSE is $438{,}746 \pm 29{,}122$: higher than the single test RMSE and with a wide spread, which shows that RMSE is dominated by a few very expensive listings and depends on which of them fall in a split. For Task B, entropy and Gini differ by only 0.0011 validation F1 (Table 4), so we use entropy with an unlimited depth. The final tree has 3,068 leaves and test F1 0.919. With matched settings, ours and scikit-learn's classification trees have 3,068 and 3,079 leaves and agree on 97.9% of test predictions; the regression trees both have 125 leaves but agree on 87.6% of predictions, so the two implementations are close in accuracy but not identical (we did not trace the difference, which may come from tie-breaking).

B. Results B: SVM

Validation selects RFF with $D = 1024$, $\gamma = 0.01$ and $\lambda = 3 \cdot 10^{-5}$. Validation F1 is 0.845, test F1 is 0.838 and test ROC-AUC is 0.924. The final objective is 0.424, the weight

Table 2. Task A (price regression). Test metrics from one fit on the training split; CV columns are mean ± std over folds on train+validation. RMSE and MAE in AZN.

Model	Test RMSE	Test MAE	Test R^2	CV RMSE	CV R^2
NumPy DecisionTree	358,672	103,507	0.661	438,746 ± 29,122	0.507 ± 0.052
sklearn DecisionTreeRegressor	358,999	103,576	0.661	438,772 ± 29,137	0.507 ± 0.052

Table 3. Task B (price tier). Test metrics plus cross-validated mean ± std (train+validation).

Model	Test F1	Test ROC-AUC	Test PR-AUC	CV F1	CV ROC-AUC	Fit (s)
NumPy DecisionTree	0.919	0.920	0.884	0.915 ± 0.003	0.916 ± 0.003	6.22
sklearn DecisionTreeClassifier	0.918	0.919	0.882	0.916 ± 0.003	0.917 ± 0.003	0.28
NumPy Pegasos SVM + RFF	0.838	0.924	0.923	0.843 ± 0.005	0.928 ± 0.002	3.62
sklearn SVC(linear)	0.839	0.923	0.912	0.843 ± 0.008	0.925 ± 0.006	125.07
sklearn SGD(hinge)	0.831	0.905	0.886	0.826 ± 0.005	0.902 ± 0.008	1.19
sklearn RBFSampler+SGD	0.833	0.924	0.921	0.840 ± 0.005	0.927 ± 0.003	1.95

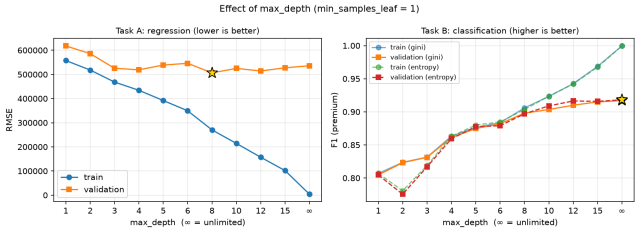


Figure 1. Validation error of our tree as a function of `max_depth`.

Table 4. Gini vs. entropy for the decision tree on Task B (validation set; each criterion at its own best `max_depth` and `min_samples_leaf`).

Criterion	Depth	Leaf	F1	AUC	Acc.	Leaves
gini	unlimited	1	0.917	0.918	0.918	3241
entropy	unlimited	1	0.918	0.919	0.919	3068

norm is 39.6, the RFF-space margin width is 0.0505, and 17,528 training examples lie on or inside the functional margin. In cross-validation our SVM obtains F1 0.843 ± 0.005, the same as scikit-learn's linear SVC (0.843 ± 0.008), while fitting in 3.4 s instead of 120 s per fold (about 36 times faster; Table 3 lists 3.6 s against 125 s for one fit on the full training split); the two solve different feature-space problems, so this is not a like-for-like solver comparison. The gain over the earlier linear run (F1 0.810) cannot be attributed to RFF alone, since the grid and the update budget also changed.

C. Bonus models

All bonus models are our NumPy code; scikit-learn is used only for comparison (Table 6). A from-scratch ridge regression and ε-SVR reach RMSE of about 583k AZN, far worse than the tree (359k), so Task A is strongly non-linear. A random forest of 30 of our trees improves Task B F1 to 0.936 (scikit-learn: 0.936) and Task A RMSE to 318,706 (scikit-learn: 343,257); the forest settings are matched only in part, so we do not claim that ours is the better implementation. AdaBoost (depth-3 trees, 100 rounds, weighted bootstrap) reaches F1 0.904 (scikit-learn: 0.900). For clustering, the first 20 principal components explain only 22.1% of the variance, and K-means ($k = 4$) and DBSCAN ($\varepsilon = 1$ km) have

Table 5. Final decision-tree configurations (chosen on validation only) and their scores.

Metric	Configuration	Validation	Test
A: RMSE	depth 8, leaf 10	468,046	358,672
A: R^2		0.511	0.661
B: F1	entropy, depth unlimited, leaf 1	0.918	0.919
B: AUC		0.919	0.920

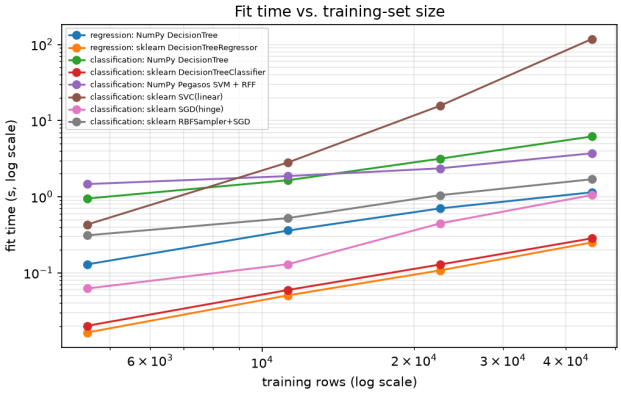


Figure 2. Fit time as the training set grows, for our models and the matched scikit-learn baselines.

adjusted Rand index of at most 0.072 against price tier and region, i.e. geography alone does not recover price tiers.

VII. Discussion and Limitations

Tree versus SVM. The tree wins on F1 (0.919 vs. 0.838) although the SVM has the higher ROC-AUC (0.924 vs. 0.920) and PR-AUC (0.923 vs. 0.884). The SVM's precision (0.768) is far below its recall (0.922), i.e. it labels too many listings premium at its default threshold. We hypothesise that a tuned threshold would close much of the F1 gap, but we did not test this.

Where the models fail. Table 7 shows the test error of the tree by true-price band. Cheap listings are over-predicted (mean signed error +29,928 AZN), and the top 1% (99 listings) are badly under-predicted (−1.48M AZN on average, MAE 2.31M): a leaf averages over its training listings and

Table 6. Bonus models (test split, scored once with the validation-selected setting).

Model	Task	Metric	Value
Ridge (ours)	A	RMSE	583,307
Ridge (ours)	A	R2	0.105
SVR (ours)	A	RMSE	583,895
SVR (ours)	A	R2	0.103
Tree (ours)	A	RMSE	358,672
Tree (ours)	A	R2	0.661
Forest (ours)	B	F1	0.936
Forest (ours)	B	ROC-AUC	0.984
sklearn RF	B	F1	0.936
Tree (ours)	B	F1	0.919
Forest (ours)	A	RMSE	318,706
Forest (ours)	A	R2	0.733
sklearn RF	A	RMSE	343,257
AdaBoost d=3 (ours)	B	F1	0.904
AdaBoost d=3 (ours)	B	ROC-AUC	0.971
sklearn AdaBoost d=3	B	F1	0.900

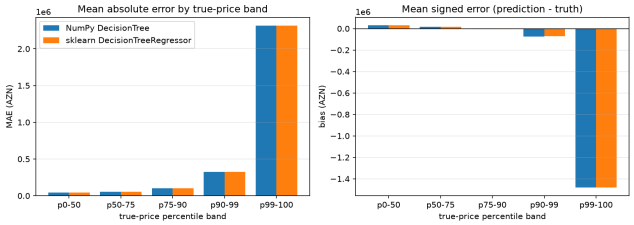


Figure 3. Task A test error of the trees by true-price band.

cannot reach extreme prices. Districts with fewer than 200 training rows have a median relative error of 0.355, against 0.177 for well-represented districts.

Limitations. The data are asking prices from a scrape, not transaction prices, and about 700 rows may be re-posted near-duplicates that we did not remove, which can slightly favour matches between train and test. Cleaning limits on price and area were fixed from the distribution, not tuned. The SVM has no projection step and no calibrated probabilities, and uses a finite-dimensional approximation of the RBF kernel. Fit times depend on the machine and the implementation; our pure-NumPy tree is about 5 to 22 times slower than scikit-learn's compiled one. AdaBoost's weighted bootstrap adds sampling variability. Results come from one seed and one split; the CV spread (±29,122 RMSE) shows that single-split RMSE differences below that size should not be over-interpreted.

VIII. Conclusion

Tree models trained from scratch match scikit-learn to within 0.002 F1 and 0.1% RMSE, and our Pegasos+RFF SVM matches a linear SVC at a fraction of the training time. On this data the tree is clearly the better classifier by F1, and bagging it into a forest helps further. Next steps are tuning the SVM decision threshold, modelling log price to reduce the luxury-listing error, and de-duplicating re-posted listings.

Tools and Acknowledgements

AI assistants were used in this project. Claude (Anthropic) wrote the first versions of the evaluation extensions

Table 7. Task A test error of our tree by true-price band (bias = prediction − truth, AZN).

Price band	*n*	MAE	Bias
0–50%	4,795	43,005	+29,928
50–75%	2,436	53,886	+16,532
75–90%	1,456	103,030	−4,906
90–99%	867	325,987	−73,114
99–100%	99	2,313,499	−1,482,189

(PR-AUC, k-fold and cross-validation), `baselines.py`, `benchmark.py`, `error_analysis.py`, the `run_all.py` pipeline, the ridge/SVR bonus and the first drafts of this report and the slides, working with Member 4, who ran, tested and committed them and reviewed the text. The Member 3 joint RFF grid and AdaBoost started from a teammate's AI-assisted draft; OpenAI Codex assisted with review, safety fixes, regression tests and real-data verification. **[Members 1 and 2: add your own AI use here, or state that you used none.]** We used scikit-learn, NumPy and matplotlib; scikit-learn appears only in baselines and bonus reference comparisons. Data: Kaggle *binaaz-sale-project* by `sehriyarmemmedli`.

References

[1] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[2] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.

[3] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[4] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems 20*, pp. 1177–1184, 2007.